

# Find differential expressed genes with 'DESeq2' in R.

Ted Ellsworth, PhD    Sofia Fertuzinhos, PhD

2026-09-28

# Bioinformatics Support at Yale

## Bioinformatics Support Hub

Computational support for every stage in the omics data lifecycle.

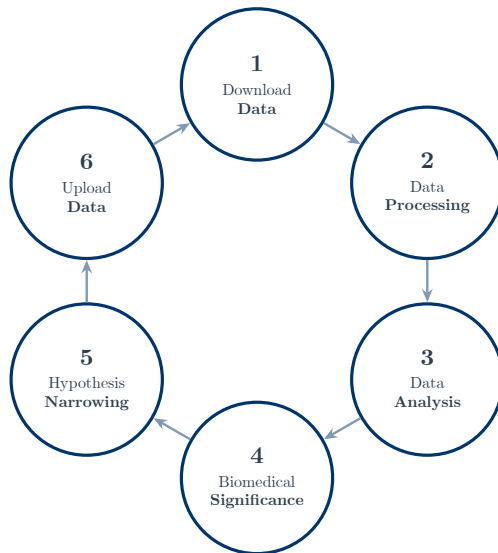
[library.medicine.yale.edu/  
research-support/bioinformatics/](https://library.medicine.yale.edu/research-support/bioinformatics/)

## StatLab Statistical Support

Statistical consulting and methods support for Yale researchers.

[library.yale.edu/help-and-research-support/  
research-support/statlab](https://library.yale.edu/help-and-research-support/research-support/statlab)

# The Lifecycle of Omics Data Analysis



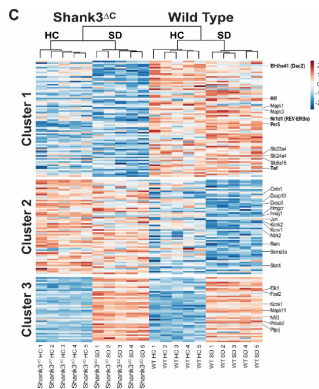
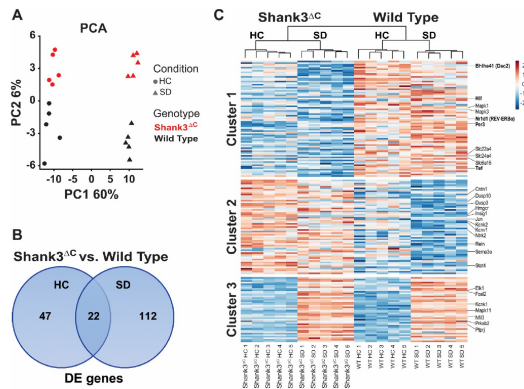
## Case Study: Shank3 and Circadian Gene Expression

RNA-seq study examining gene expression in the prefrontal cortex:

- **Tissue:** Brain — prefrontal cortex
- **Animal:** Mouse, 9 weeks old
- **Genotype:**
  - Shank3<sup>ΔC</sup> ( $n = 10$ )
  - Wild type ( $n = 10$ )
- **Condition:**
  - Normal home cage (5 per genotype)
  - 5 hr sleep deprivation (5 per genotype)

This is a **2 × 2 factorial design**: two genotypes crossed with two conditions, yielding four groups.

# Results: Design Shapes What You Find



How does the **design** of the analysis impact results and interpretation?

Figure 1: PCA, Venn diagram, and heatmap from Shank3 study

# Results: Design Shapes What You Find

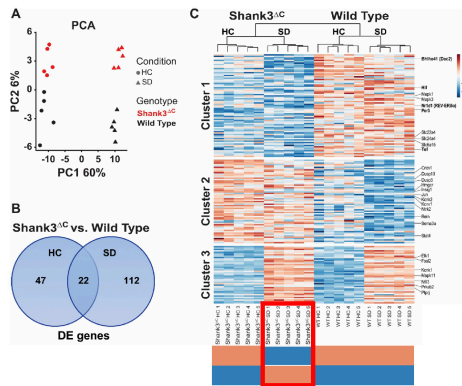


Figure 2: PCA, Venn diagram, and heatmap from Shank3 study

A simple Genotype-only design misses genes **specifically affected** in Shank3<sup>ΔC</sup> / sleep-deprived mice.

The Venn diagram shows 22 DE genes shared between conditions, but 112 unique to sleep deprivation.

# Data Preparation

# Workflow: Cleaning Data for DESeq2

## Download from GREIN

1. Navigate to `ilincs.org/apps/grein/`
2. Enter GEO accession (e.g., GSE113754)
3. Download **raw counts .csv**
4. Download **metadata .csv**



# Step 1: Downloading Data (GREIN)

## GREIN (GEO RNA-seq Experiments Interactive Navigator)

- Search by **GEO series accession** (e.g., GSE113754)
- Pre-aligned, gene-level raw counts ready to download
- Metadata table downloadable alongside counts
- Output: .csv files compatible with DESeq2

[ilincs.org/apps/grein/](http://ilincs.org/apps/grein/)

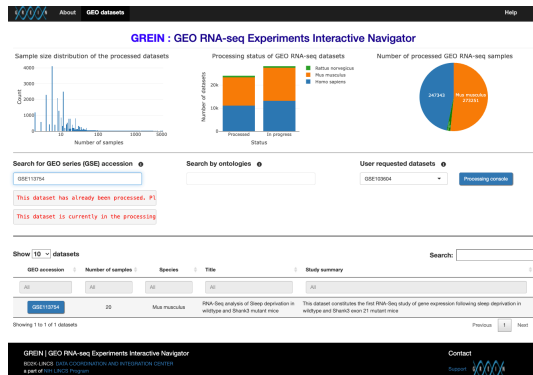


Figure 3: GREIN interface

# Raw Counts: Downloading from GREIN

**Selected study**  
GSE113754

**Data type**  
☒ Raw ☐ Normalized

**Number of samples to show**  
20

[Show counts table](#)

[Download data](#)

☒ Gene level ☐ Transcript level

**Counts table**

Select any row to see boxplot of the selected gene below the table.

Show 8 genes Search:

| Gene symbol        | GSM3118997 | GSM3118998 | GSM3118999 | GSM3119000 | GSM3119001 | 0    |
|--------------------|------------|------------|------------|------------|------------|------|
| ENSMUSG00000000001 | Gna3       | 1689       | 1278       | 2153       | 828        | 1146 |
| ENSMUSG00000000003 | Pben       | 0          | 0          | 0          | 0          | 0    |
| ENSMUSG00000000028 | Cdc45      | 92         | 71         | 110        | 52         | 60   |
| ENSMUSG00000000031 | H19        | 2          | 4          | 8          | 2          | 2    |
| ENSMUSG00000000037 | Somf2      | 77         | 62         | 131        | 35         | 73   |
| ENSMUSG00000000049 | Apo1       | 17         | 20         | 26         | 4          | 13   |
| ENSMUSG00000000056 | Narf       | 3100       | 2445       | 3507       | 1366       | 1808 |
| ENSMUSG00000000058 | Cav2       | 2146       | 1225       | 2854       | 1305       | 1574 |

Showing 1 to 8 of 28,886 genes

Previous 1 2 3 4 5 ... 3611 Next

GREIN | GEO RNA-seq Experiments Interactive Navigator  
BDK4-UNCS DATA COORDINATION AND INTEGRATION CENTER

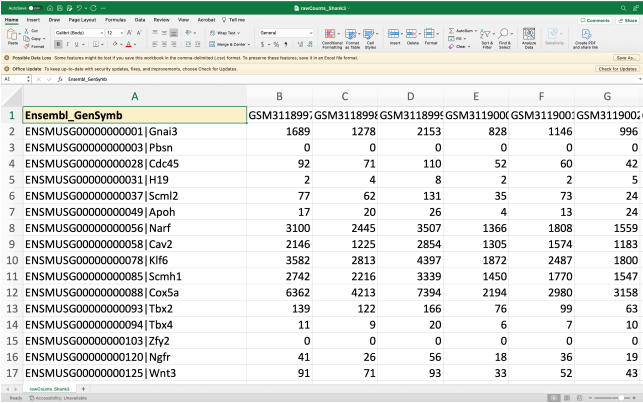
Contact

The **Counts table** tab shows gene-level raw counts. Rows are genes (Ensembl ID + gene symbol); columns are samples.

Select **Raw** data type, then click **Download data** to save as **.csv**.

# Raw Counts: The CSV File

The downloaded file opens in Excel (or can be read directly into R). The first column combines the Ensembl gene ID with the gene symbol.



|    | A                        | B         | C         | D         | E         | F         | G         |
|----|--------------------------|-----------|-----------|-----------|-----------|-----------|-----------|
| 1  | <b>Ensembl_GenSymb</b>   | GSM311899 | GSM311899 | GSM311899 | GSM311900 | GSM311900 | GSM311900 |
| 2  | ENSMUSG0000000001 Gnai3  | 1689      | 1278      | 2153      | 828       | 1146      | 996       |
| 3  | ENSMUSG0000000003 Pbsn   | 0         | 0         | 0         | 0         | 0         | 0         |
| 4  | ENSMUSG00000000028 Cdc45 | 92        | 71        | 110       | 52        | 60        | 42        |
| 5  | ENSMUSG00000000031 H19   | 2         | 4         | 8         | 2         | 2         | 5         |
| 6  | ENSMUSG00000000037 Scml2 | 77        | 62        | 131       | 35        | 73        | 24        |
| 7  | ENSMUSG00000000049 Apoh  | 17        | 20        | 26        | 4         | 13        | 24        |
| 8  | ENSMUSG00000000056 Narf  | 3100      | 2445      | 3507      | 1366      | 1808      | 1559      |
| 9  | ENSMUSG00000000058 Cav2  | 2146      | 1225      | 2854      | 1305      | 1574      | 1183      |
| 10 | ENSMUSG00000000078 Klf6  | 3582      | 2813      | 4397      | 1872      | 2487      | 1800      |
| 11 | ENSMUSG00000000085 Scmh1 | 2742      | 2216      | 3339      | 1450      | 1770      | 1547      |
| 12 | ENSMUSG00000000088 Cox5a | 6362      | 4213      | 7394      | 2194      | 2980      | 3158      |
| 13 | ENSMUSG00000000093 Tbx2  | 139       | 122       | 166       | 76        | 99        | 63        |
| 14 | ENSMUSG00000000094 Tbx4  | 11        | 9         | 20        | 6         | 7         | 10        |
| 15 | ENSMUSG00000000103 Zfy2  | 0         | 0         | 0         | 0         | 0         | 0         |
| 16 | ENSMUSG00000000120 Ngfr  | 41        | 26        | 56        | 18        | 36        | 19        |
| 17 | ENSMUSG00000000125 Wnt3  | 91        | 71        | 93        | 33        | 52        | 43        |

Figure 4: rawCounts\_Shank3.csv opened in Excel

# Metadata: Downloading from GREIN

The **Metadata** tab lists sample-level characteristics. Select **Filtered metadata** and check the variables you need before downloading.

**Selected study**

GSE113754

**Select properties**

- ☒ characteristics
- ☒ genotype
- ☒ genotype.ch1

**Types of metadata:**

- ☒ Filtered metadata
- ☐ Full metadata

[Download metadata](#)

**Description Metadata Counts table QC report Visualization**

Show 10 samples Search:

|            | characteristics                                               | genotype      | genotype.ch1  |
|------------|---------------------------------------------------------------|---------------|---------------|
| GSM3118997 | Wildtype homecage control harvested at ZT hour 5, replicate 1 | WT            | WT            |
| GSM3118998 | Wildtype homecage control harvested at ZT hour 5, replicate 2 | WT            | WT            |
| GSM3118999 | Wildtype homecage control harvested at ZT hour 5, replicate 3 | WT            | WT            |
| GSM3119000 | Wildtype homecage control harvested at ZT hour 5, replicate 4 | WT            | WT            |
| GSM3119001 | Wildtype homecage control harvested at ZT hour 5, replicate 5 | WT            | WT            |
| GSM3119002 | Shank3 mutant homecage control at ZT hour 5, replicate 1      | Shank3 Mutant | Shank3 Mutant |
| GSM3119003 | Shank3 mutant homecage control at ZT hour 5, replicate 2      | Shank3 Mutant | Shank3 Mutant |
| GSM3119004 | Shank3 mutant homecage control at ZT hour 5, replicate 3      | Shank3 Mutant | Shank3 Mutant |
| GSM3119005 | Shank3 mutant homecage control at ZT hour 5, replicate 4      | Shank3 Mutant | Shank3 Mutant |
| GSM3119006 | Shank3 mutant homecage control at ZT hour 5, replicate 5      | Shank3 Mutant | Shank3 Mutant |

Showing 1 to 10 of 20 samples

Previous 1 2 Next

GREIN | GEO RNA-seq Experiments Interactive Navigator  
BD2K-LINCS DATA COORDINATION AND INTEGRATION CENTER  
a part of NIH LINCS Program

Contact  
Support

Figure 5: GREIN metadata tab — GSE113754

# Metadata: The CSV File

The metadata file records the experimental grouping for each sample. The combined **Geno\_Cond** column captures all four groups.

|    | A          | B        | C            | D                     | E | F | G | H | I | J | K |
|----|------------|----------|--------------|-----------------------|---|---|---|---|---|---|---|
| 1  | Sample     | genotype | Condition    | Geno_Cond             |   |   |   |   |   |   |   |
| 2  | GSM3118997 | WT       | _normal      | WT_normal             |   |   |   |   |   |   |   |
| 3  | GSM3118998 | WT       | _normal      | WT_normal             |   |   |   |   |   |   |   |
| 4  | GSM3118999 | WT       | _normal      | WT_normal             |   |   |   |   |   |   |   |
| 5  | GSM3119000 | WT       | _normal      | WT_normal             |   |   |   |   |   |   |   |
| 6  | GSM3119001 | WT       | _normal      | WT_normal             |   |   |   |   |   |   |   |
| 7  | GSM3119002 | Shank3   | _normal      | Shank3_normal         |   |   |   |   |   |   |   |
| 8  | GSM3119003 | Shank3   | _normal      | Shank3_normal         |   |   |   |   |   |   |   |
| 9  | GSM3119004 | Shank3   | _normal      | Shank3_normal         |   |   |   |   |   |   |   |
| 10 | GSM3119005 | Shank3   | _normal      | Shank3_normal         |   |   |   |   |   |   |   |
| 11 | GSM3119006 | Shank3   | _normal      | Shank3_normal         |   |   |   |   |   |   |   |
| 12 | GSM3119007 | WT       | _sleep_depri | WT_sleep_deprived     |   |   |   |   |   |   |   |
| 13 | GSM3119008 | WT       | _sleep_depri | WT_sleep_deprived     |   |   |   |   |   |   |   |
| 14 | GSM3119009 | WT       | _sleep_depri | WT_sleep_deprived     |   |   |   |   |   |   |   |
| 15 | GSM3119010 | WT       | _sleep_depri | WT_sleep_deprived     |   |   |   |   |   |   |   |
| 16 | GSM3119011 | WT       | _sleep_depri | WT_sleep_deprived     |   |   |   |   |   |   |   |
| 17 | GSM3119012 | Shank3   | _sleep_depri | Shank3_sleep_deprived |   |   |   |   |   |   |   |
| 18 | GSM3119013 | Shank3   | _sleep_depri | Shank3_sleep_deprived |   |   |   |   |   |   |   |
| 19 | GSM3119014 | Shank3   | _sleep_depri | Shank3_sleep_deprived |   |   |   |   |   |   |   |
| 20 | GSM3119015 | Shank3   | _sleep_depri | Shank3_sleep_deprived |   |   |   |   |   |   |   |

Figure 6: metadata\_Shank3.csv opened in Excel

# DESeq2 Workflow Overview

## Analysis steps in R / Positron:

1. Import `.csv` files with `read.csv()`
2. Load the DESeq2 package (install BiocManager first)
3. Assemble `DESeqDataSet` and specify design
4. Run `DESeq()` — normalization + model fitting
5. Extract normalized counts and results table
6. Visualize: PCA, MA plot, `plotCounts()`



Figure 7: DESeq2 Bioconductor package

# R Walkthrough

# Getting Started in Positron

1. `github.com/YaleLibraryStatLab/differential-expression`: **Code** → **Download ZIP**, then unzip
2. Positron: **File** → **Open Folder...** → select `differential-expression`
3. Open `scripts/DESeq2.R`
4. Check the working directory: `getwd()` should end in `differential-expression`

## Best Practice: Use `here()` Instead of Hard Paths

Hard-coded paths break when you move your project or share it. The `here` package builds paths relative to your project root automatically: `library(here)`  
`read.csv(here("data", "raw", "rawCounts_Shank3.csv"))`



# Importing Data in R

```
library(here)

rawc <- read.csv(
  here("data", "raw", "rawCounts_Shank3.csv"),
  header = T, row.names = "Ensembl_GenSymb")

info <- read.csv(
  here("data", "raw", "metadata_Shank3.csv"),
  header = T, stringsAsFactors = T)

# Specify the baseline for comparisons
info$genotype <- relevel(info$genotype, "WT")
info$Condition <- relevel(info$Condition, "_normal")
```

## Note

here() paths start at the workshop folder you opened in Positron.

## Why relevel()?

DESeq2 uses the **first factor level** as the reference for fold-change calculations — ensures the control group is the baseline.

# Assembling the DESeqDataSet

```
if (!require("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
BiocManager::install("DESeq2")
library(DESeq2)

## Assemble DESeqDataSet
Shank3 <- DESeqDataSetFromMatrix(
  countData = rawc,
  colData    = info,
  design     = ~
)
```

The formula specifies which metadata column(s) model the variation in gene counts.  
Choosing the right design is the most consequential decision in the analysis.

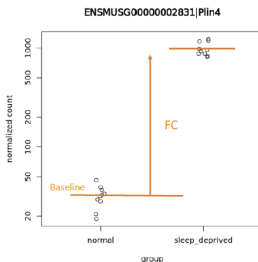
# Metadata and the Model Matrix

```
dds <- DESeqDataSetFromMatrix(countData = countData, colData = metadata, design = ~ Condition)
dds <- DESeq(dds)
```

```
model.matrix(~Condition, data=metadata)
```

| Sample  | Condition |             |                   |
|---------|-----------|-------------|-------------------|
| <fct>   | <fct>     | (Intercept) | ConditionSleepDep |
| sample1 | Normal    | 1           | 0                 |
| sample2 | Normal    | 1           | 0                 |
| sample3 | SleepDep  | 1           | 1                 |
| sample4 | SleepDep  | 1           | 1                 |

Design = ~ (baseline) + Condition



The key to all designs is how we **organize the metadata** and **design the model matrix**.

The formula ~ (baseline) + Condition tells DESeq2 how to model counts per gene. (*Credit: Hugo Tavares*)

## Design Matrices & Linear Models

# What is a Design Matrix?

The *design matrix*  $X$  translates your experimental design into a form the model can use

- **Rows** = samples (observations)
- **Columns** = model parameters (intercept, group effects, batch effects, ...)

## Design Matrix

“The design matrix is the mathematical encoding of your experimental design. The choice of formula is not a software concern – it determines what questions your analysis can answer.” — adapted from Law et al. (2020)

# What is a Linear Model?

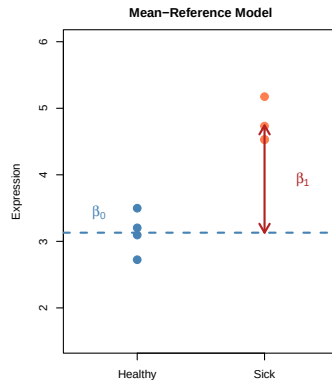
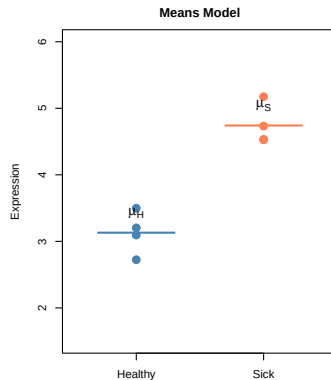
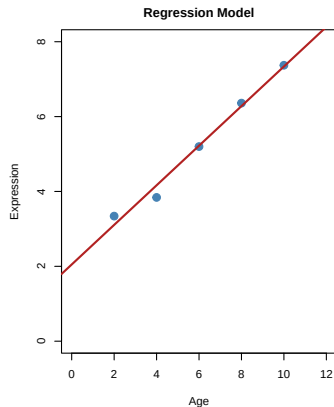
$$y = X\beta + \varepsilon$$

- $y$  — response vector (observed counts for one gene, across samples)
- $X$  — design matrix encoding the experimental structure
- $\beta$  — parameter vector (effects we want to estimate)
- $\varepsilon$  — residual / noise term

This framework generalizes: we fit one model per gene, then correct for multiple testing across all ~20,000 tests.

```
model.matrix(~ condition, data = sample_info)
```

# The Three Basic Model Types



Three fundamental model types (Law et al. 2020): regression, means, and mean-reference.



## Reading `model.matrix()` Output

```
model.matrix(~ condition, data = sample_info)
```

|        | (Intercept) | conditiontreated |
|--------|-------------|------------------|
| ctrl_1 | 1           | 0                |
| ctrl_2 | 1           | 0                |
| ctrl_3 | 1           | 0                |
| trt_1  | 1           | 1                |
| trt_2  | 1           | 1                |
| trt_3  | 1           | 1                |

- Intercept = mean of the **reference** level (control)
- `conditiontreated` = difference: treated – control

# Means vs. Mean-Reference: Two Parameterizations

## Means model ( $\sim 0 + \text{condition}$ )

- $\beta_1 = \mu_{\text{ctrl}}, \beta_2 = \mu_{\text{trt}}$
- Test:  $\beta_2 - \beta_1 = 0$
- Direct group means; contrasts required

## Mean-reference model ( $\sim \text{condition}$ )

- $\beta_0 = \mu_{\text{ctrl}}$
- $\beta_1 = \mu_{\text{trt}} - \mu_{\text{ctrl}}$
- Test:  $\beta_1 = 0$  — R's default

Both are **mathematically equivalent** — identical fitted values and residuals.

## Setting the Reference Level

```
# Set reference level at creation
sample_info$condition <- factor(sample_info$condition,
                                levels = c("control", "treated"))

# Or relevel an existing factor
sample_info$condition <- relevel(sample_info$condition,
                                ref = "control")
```

### Warning

R uses **alphabetical order** by default. Always set the reference level explicitly so your log fold changes have the direction you expect.

## Treatments vs. Control

4-group scenario: CTL, Treatment I, II, III — use `relevel()` to set CTL as reference.

```
sample_info$treatment <- relevel(sample_info$treatment, ref = "CTL")
model.matrix(~ treatment, data = sample_info)
```

|          | (Intercept) | treatmentI | treatmentII | treatmentIII |
|----------|-------------|------------|-------------|--------------|
| CTL_1    | 1           | 0          | 0           | 0            |
| TrtI_1   | 1           | 1          | 0           | 0            |
| TrtII_1  | 1           | 0          | 1           | 0            |
| TrtIII_1 | 1           | 0          | 0           | 1            |

Each treatment column gives the treatment-vs-control difference directly.

# All Pairwise Comparisons: Means Model

Same 4 groups with  $\sim 0 + \text{treatment}$  — each column estimates a group mean.

|          | treatCTL | treatI | treatII | treatIII |
|----------|----------|--------|---------|----------|
| CTL_1    | 1        | 0      | 0       | 0        |
| TrtI_1   | 0        | 1      | 0       | 0        |
| TrtII_1  | 0        | 0      | 1       | 0        |
| TrtIII_1 | 0        | 0      | 0       | 1        |

Any pairwise comparison is then a contrast: e.g., Treatment I vs II: `contrast = c("treatment", "I", "II")`

# Contrasts

## Contrast

A contrast is a linear combination of model parameters specified by a weight vector.

Examples (means model  $\sim 0 + \text{treatment}$ ):

- Treatment I vs. Control: weights  $(-1, 1, 0, 0)$
- Average of all treatments vs. Control: weights  $(-1, \frac{1}{3}, \frac{1}{3}, \frac{1}{3})$
- Groups I+II vs. III+CTL: weights  $(-\frac{1}{2}, \frac{1}{2}, \frac{1}{2}, -\frac{1}{2})$

```
results(dds, contrast = c(0, 1/3, 1/3, 1/3))
```

# Interaction Designs

```
# ~ genotype * treatment expands to:  
# ~ genotype + treatment + genotype:treatment  
model.matrix(~ genotype * treatment, data = sample_info)
```

|          | (Int.) | genoMut | trtStim | genoMut:trtStim |
|----------|--------|---------|---------|-----------------|
| WT_ctrl  | 1      | 0       | 0       | 0               |
| WT_stim  | 1      | 0       | 1       | 0               |
| Mut_ctrl | 1      | 1       | 0       | 0               |
| Mut_stim | 1      | 1       | 1       | 1               |

The interaction column equals 1 **only** when both non-reference levels are present.

# Interaction as Synergy or Additivity

$$\delta = C - A - B$$

where  $A$  = effect of treatment 1,  $B$  = effect of treatment 2,  $C$  = combined effect.

- $\delta > 0$ : **synergistic** — combined exceeds sum of parts
- $\delta = 0$ : **additive** — no interaction
- $\delta < 0$ : **antagonistic** — combined less than sum of parts

The interaction coefficient in `~ treat1 * treat2` directly estimates  $\delta$ .

## Reference-Level Dependency

The interaction coefficient  $\delta$  is defined **relative to the reference levels** of both factors. Changing the reference level of either factor changes the value of  $\delta$ . Always report which levels are being compared when interpreting an interaction term.



# The Group Variable Shortcut

## Replication Required

Interaction terms require adequate replication in **all** factor-level combinations. With unbalanced designs, the group variable approach is often simpler.

Merge factors into a single group variable — all pairwise contrasts become direct.

```
dds$group <- factor(paste0(dds$tissue, "_", dds$celltype))
design(dds) <- ~ 0 + group
```

|           | groupLUNG_B | groupBRAIN_B | groupLUNG_T | groupBRAIN_T |
|-----------|-------------|--------------|-------------|--------------|
| LUNG_B_1  | 1           | 0            | 0           | 0            |
| BRAIN_B_1 | 0           | 1            | 0           | 0            |
| LUNG_T_1  | 0           | 0            | 1           | 0            |
| BRAIN_T_1 | 0           | 0            | 0           | 1            |

B vs T within lung: `contrast = c("group", "LUNG_B", "LUNG_T")`. Recommended by Law et al. (2020) and the DESeq2 vignette.

## Paired and Repeated Measures

Scenario: 6 mice (3 per treatment), each measured at two time points.

`~ treatment + time` may fail when mice are **nested** within treatment. Instead, model individual baselines with a numeric design matrix:

```
mm <- model.matrix(~ 0 + mouse + treatment_time,  
                  data = sample_info)  
design(dds) <- mm  
dds <- DESeq(dds)
```

# Multi-Factor Designs

```
model.matrix(~ batch + condition, data = sample_info)
```

|          | (Intercept) | batchB | conditiontreated |
|----------|-------------|--------|------------------|
| A_ctrl_1 | 1           | 0      | 0                |
| A_ctrl_2 | 1           | 0      | 0                |
| A_trt_1  | 1           | 0      | 1                |
| B_ctrl_1 | 1           | 1      | 0                |
| B_trt_1  | 1           | 1      | 1                |
| B_trt_2  | 1           | 1      | 1                |

The condition effect is estimated **after removing** batch differences.

# The Design Matrix Absorbs Nuisance Variation

- Adding **batch** partitions variance into batch + condition + residual
- The condition coefficient is estimated **conditional on** batch
- **Without batch:** estimate contaminated by batch-to-batch variation
- **With batch:** “cleaned” estimate — only within-batch treatment difference

## Rule of Thumb

Include a variable in the design if PCA shows it drives a principal component. The design matrix will absorb that variation (Law et al. 2020).

# Where the Linear Model Breaks Down

The standard linear model  $y = X\beta + \varepsilon$  assumes conditions that count data routinely violates:

- **Normal errors** — counts are non-negative integers; residuals for low-count genes are right-skewed, not normal
- **Constant variance** — variance in RNA-seq grows with the mean; high-abundance genes show far more raw spread
- **Unbounded predictions** — the linear predictor  $X\beta$  can return negative fitted counts

# From Linear Models to GLMs

Generalized linear models add a **link function**  $g$ :

$$g(\mu) = X\beta \quad \implies \quad \log(\mu) = X\beta$$

- For count data we use the **log link**:  $\mu = e^{X\beta}$
- Effects are **multiplicative** on the original scale
- DESeq2 coefficients are on the **natural log scale** internally

# Why Negative Binomial?

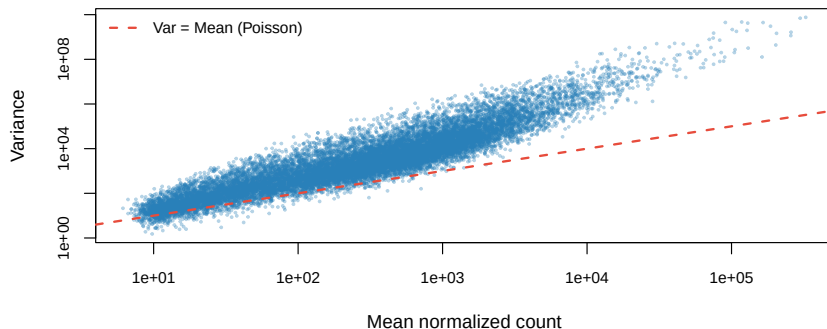
## Negative Binomial Distribution

$K_{ij} \sim \text{NB}(\mu_{ij}, \alpha_i)$ , where  $\mu_{ij}$  is the mean count for gene  $i$  in sample  $j$ , and  $\alpha_i$  is the gene-specific **dispersion** parameter.

- **Poisson** assumes  $\text{Var} = \mu$  — too restrictive for biological data
- Real RNA-seq counts exhibit **overdispersion**: variance  $>$  mean
- The NB adds dispersion  $\alpha$  capturing extra biological variability

$$\text{Var}(K_{ij}) = \mu_{ij} + \alpha_i \cdot \mu_{ij}^2$$

# Variance Grows with the Mean



Each point is a gene. Observed variance exceeds the Poisson expectation at every expression level.



## Coefficients: Internal Scale vs. Reported Output

DESeq2 fits the model on the natural log scale, but reports results on  $\log_2$ :

$$\log_2 \text{FC} = \frac{\beta}{\log(2)}$$

A raw coefficient  $\beta_1 = 0.5$  becomes  $0.5/0.693 \approx 0.72$  in the output — a  $2^{0.72} \approx 1.65$ -fold change.

Always back-calculate from `log2FoldChange` in the results table, not from the raw GLM coefficient.

# Checking Your Design

```
mm <- model.matrix(~ batch + condition, data = sample_info)
qr(mm)$rank    # should equal ncol(mm)
ncol(mm)       # number of parameters
```

## Rank Deficiency

If  $\text{rank} < \text{ncol}$ , columns are linearly dependent — some factor is **perfectly confounded** with another. This is a **design problem**, not a software problem. DESeq2 will refuse to run, and rightly so.

## Design Challenge

## Exercise 1: Choose the Right Formula

**Experiment:** 6 samples, one factor

| Sample | Condition |
|--------|-----------|
| ctrl_1 | control   |
| ctrl_2 | control   |
| ctrl_3 | control   |
| trt_1  | treated   |
| trt_2  | treated   |
| trt_3  | treated   |

**Goal:** compare treated vs. control

Which design formula?

- A. `~ 0 + condition`
- B. `~ condition`
- C. `~ condition + sample`
- D. `~ 1`

## Exercise 1: Answer

| Sample | Condition |
|--------|-----------|
| ctrl_1 | control   |
| ctrl_2 | control   |
| ctrl_3 | control   |
| trt_1  | treated   |
| trt_2  | treated   |
| trt_3  | treated   |

A. `~ 0 + condition` — means model, valid but needs explicit contrast

### B. `~ condition`

Mean-reference model: intercept = control mean; `conditiontreated` = fold change directly. R's default parameterization.

C. Over-specified — sample absorbs all variance

D. Intercept only — no group comparison possible

## Exercise 2: Choose the Right Formula

**Experiment:** 2 batches, treated vs. control in each

| Sample   | Batch | Condition |
|----------|-------|-----------|
| A_ctrl_1 | A     | control   |
| A_ctrl_2 | A     | control   |
| A_trt_1  | A     | treated   |
| B_ctrl_1 | B     | control   |
| B_trt_1  | B     | treated   |
| B_trt_2  | B     | treated   |

**Goal:** treatment effect adjusted for batch

Which design formula?

- A. ~ condition
- B. ~ batch
- C. ~ batch + condition
- D. ~ batch \* condition

## Exercise 2: Answer

| Sample   | Batch | Condition |
|----------|-------|-----------|
| A_ctrl_1 | A     | control   |
| A_ctrl_2 | A     | control   |
| A_trt_1  | A     | treated   |
| B_ctrl_1 | B     | control   |
| B_trt_1  | B     | treated   |
| B_trt_2  | B     | treated   |

A. Ignores batch — condition estimate contaminated

B. Models batch only — not the goal

### C. `~ batch + condition`

Batch absorbs technical variation;  
`conditiontreated` estimates the treatment effect within each batch.

D. Tests whether treatment effect *differs* by batch — not needed here

## Exercise 3: Choose the Right Formula

**Experiment:** one control + three treatments

| Sample | Treatment  |
|--------|------------|
| CTL_1  | CTL        |
| CTL_2  | CTL        |
| TrtA_1 | TreatmentA |
| TrtA_2 | TreatmentA |
| TrtB_1 | TreatmentB |
| TrtB_2 | TreatmentB |

**Goal:** each treatment vs. CTL as the reference

Which design formula?

- A. `~ 0 + treatment`
- B. `~ treatment`  
(after `relevel(ref = "CTL")`)
- C. `~ treatment - 1`
- D. `~ treatment + sample`



## Exercise 3: Answer

| Sample | Treatment  |
|--------|------------|
| CTL_1  | CTL        |
| CTL_2  | CTL        |
| TrtA_1 | TreatmentA |
| TrtA_2 | TreatmentA |
| TrtB_1 | TreatmentB |
| TrtB_2 | TreatmentB |

A. Means model — valid, but requires explicit contrasts for each vs. CTL

### B. `~ treatment`

After `relevel(ref = "CTL")`, each coefficient is TreatmentX — CTL directly. No explicit contrasts needed.

C. Equivalent to A — removes the intercept

D. Overfit — sample is nested within treatment; model matrix is rank-deficient

## Exercise 4: Choose the Right Formula

**Experiment:** 2 genotypes  $\times$  2 treatments  
(2  $\times$  2 factorial)

| Sample   | Genotype | Treatment |
|----------|----------|-----------|
| WT_ctrl  | WT       | Control   |
| WT_drug  | WT       | Drug      |
| Mut_ctrl | Mutant   | Control   |
| Mut_drug | Mutant   | Drug      |

**Goal:** detect genes where the drug effect  
*differs* between genotypes

Which design formula?

- A. `~ genotype + treatment`
- B. `~ genotype`
- C. `~ genotype * treatment`
- D. `~ treatment`

## Exercise 4: Answer

| Sample   | Genotype | Treatment |
|----------|----------|-----------|
| WT_ctrl  | WT       | Control   |
| WT_drug  | WT       | Drug      |
| Mut_ctrl | Mutant   | Control   |
| Mut_drug | Mutant   | Drug      |

A. Additive only — assumes drug effect is the same in both genotypes

B. Genotype main effect only — ignores treatment entirely

### C. `~ genotype * treatment`

Expands to `~ genotype + treatment + genotype:treatment`. The interaction term tests whether the drug effect *differs by genotype*.

D. Treatment main effect only — ignores genotype

## Exercise 5: The Shank3 Study

**Our case study:** 2 genotypes  $\times$  2 conditions,  $n = 5$  per group

| Genotype          | Condition      | $n$ |
|-------------------|----------------|-----|
| Shank3 $\Delta^C$ | Normal         | 5   |
| Shank3 $\Delta^C$ | Sleep_deprived | 5   |
| Wild Type         | Normal         | 5   |
| Wild Type         | Sleep_deprived | 5   |

**Goal:** find genes specifically altered in Shank3 mice under sleep deprivation

Which design formula?

- A. `~ genotype`
- B. `~ Condition`
- C. `~ genotype + Condition`
- D. `~ genotype + Condition + genotype:Condition`

## Case Study: Complete DESeq2 Setup

**Our case study:** 2 genotypes  $\times$  2 conditions,  
 $n = 5$  per group

| Genotype                                | Condition      | $n$ |
|-----------------------------------------|----------------|-----|
| Shank3 <sup><math>\Delta^C</math></sup> | Normal         | 5   |
| Shank3 <sup><math>\Delta^C</math></sup> | Sleep_deprived | 5   |
| Wild Type                               | Normal         | 5   |
| Wild Type                               | Sleep_deprived | 5   |

A. `~ genotype` — misses the condition effect

B. `~ Condition` — misses the genotype effect

C. Additive only — assumes genotype and condition effects are independent

**D. `~ genotype + Condition + genotype:Condition`**

The interaction term captures genes whose response to sleep deprivation *differs* between Shank3 and WT mice.

## Case Study: Complete DESeq2 Setup in R

```
## Assemble DESeqDataSet
Shank3 <- DESeqDataSetFromMatrix(
  countData = rawc,
  colData    = info,
  design     = ~ genotype + Condition +
              genotype:Condition
)
```

## Running DESeq2: Shank3 Case Study

## Exploratory Analysis: rlog and plotPCA()

```
log_DEX_Shank3 <- rlog(DEX_Shank3)
plotPCA(log_DEX_Shank3, intgroup = "Geno_Cond")
```

- `rlog()` applies a regularized log transformation, stabilizing variance across genes
- Use `intgroup` to colour samples by an experimental group
- The `plotPCA()` function from DESeq2 returns a `ggplot2` object — customizable

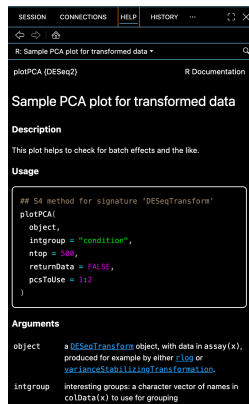


Figure 8: `plotPCA` documentation



# PCA Interpretation

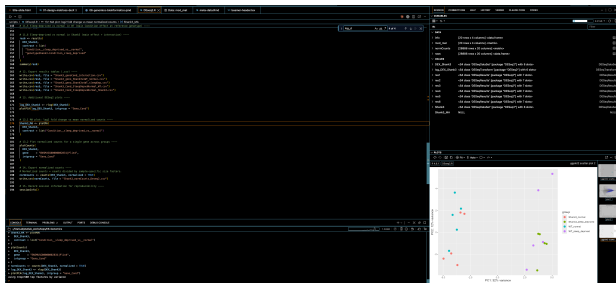


Figure 9: PCA of the Shank3 dataset coloured by group

## Key Observation

The dominant source of variation (PC1: 32%) is **housing condition** — WT and Shank3 samples are *mixed* within the normal and sleep-deprived clouds.

This means **condition** should come first in the design formula, ahead of genotype.

## Re-designing Based on the PCA

PCA shows condition drives more variance than genotype — place it first in the formula so DESeq2 reports its coefficient first via `resultsNames()`.

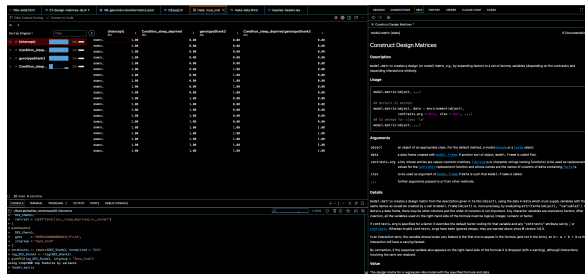
```
Shank3 <- DESeqDataSetFromMatrix(  
  countData = rawc,  
  colData    = info,  
  design     = ~ Condition + genotype +  
               Condition:genotype  
)
```

The **last** term in the formula is the default coefficient returned by `results()`. Placing the interaction last keeps it as the default output — the comparison of greatest interest.

# Understanding the Model Matrix

Use `model.matrix()` to inspect exactly what DESeq2 is fitting.

```
mod_mat <- model.matrix(design(DEX_Shank3),
                        colData(DEX_Shank3))
```



Four columns confirm the design:

- (Intercept) = WT, Normal baseline
- Condition\_sleep\_deprived
- genotypeShank3
- Condition\_sleep\_deprived:genotypeShank3

Figure 10: Resulting model matrix (first rows)

## Extracting Results: `resultsNames()`

```
resultsNames(DEX_Shank3)
# [1] "Intercept"
# [2] "Condition__sleep_deprived_vs__normal"
# [3] "genotype_Shank3_vs_WT"
# [4] "genotypeShank3.Condition_sleep_deprived"
```

### Four Estimable Comparisons

1. Condition: sleep-deprived vs. normal (*in WT*)
2. Genotype: Shank3 vs. WT (*in normal conditions*)
3. Interaction: does the Condition effect differ in Shank3?

The default `results(DEX_Shank3)` returns the **last** coefficient — the interaction term.

# Comparison 1 & 2: Genotype Effect

```

115 # 11. Extract DESeq2 results for specific comparisons ----
116
117 # 11.1 Default results() output (depends on the last term in the design) ----
118 res1 <- results(DEX_Shank3)
119 summary(res1)
120
121 # 11.2 Interaction term: genotype x condition ----
122 res2 <- results(
123   DEX_Shank3,
124   contrast = list("genotypeShank3.Condition_sleep_deprived")
125 )
126 summary(res2)
127
128 # 11.3 Shank3 vs WT (main genotype effect at the reference condition level) ----
129 res3 <- results(
130   DEX_Shank3,
131   contrast = list("genotype_Shank3_vs_WT")
132 )
133 summary(res3)
134
135 # 11.4 Shank3 vs WT in sleep-deprived condition (main effect + interaction) ----
136 res4 <- results(
137   DEX_Shank3,
138   contrast = list(
139     "genotype_Shank3_vs_WT",
140     "genotypeShank3.Condition_sleep_deprived"
141   )
142 )
143 summary(res4)
144
145 # 11.5 Sleep-deprived vs normal in WT (main condition effect at reference genotype) ----
146 res5 <- results(
147   DEX_Shank3,
148   contrast = list("Condition_sleep_deprived_vs_normal")
149 )
150 summary(res5)
151
152 # 11.6 Sleep-deprived vs normal in Shank3 (main effect + interaction) ----
153 res6 <- results(

```

Figure 11: Results extraction code

Shank3 vs WT in *normal* conditions:

```
res3 <- results(DEX_Shank3,
  contrast = list(
    "genotype_Shank3_vs_WT")
)
```

Shank3 vs WT in *sleep-deprived* conditions (add interaction):

```
res4 <- results(DEX_Shank3,
  contrast = list(
    "genotype_Shank3_vs_WT",
    "genotypeShank3.Condition_sleep_deprived"
  ))
```

## Comparison 3 & 4: Condition Effect

Sleep-deprived vs normal in *WT* mice:

```
res5 <- results(DEX_Shank3,  
  contrast = list("Condition__sleep_deprived_vs__normal"))
```

Sleep-deprived vs normal in *Shank3* mice (add interaction):

```
res6 <- results(DEX_Shank3,  
  contrast = list(  
    "Condition__sleep_deprived_vs__normal",  
    "genotypeShank3.Condition_sleep_deprived"  
  ))
```

### The Interaction Pattern

Combining a main-effect coefficient with the interaction coefficient "shifts" the comparison from the reference genotype to the other. See: [bioconductor.org](https://bioconductor.org/packages/devel/bioc/vignettes/DESeq2/doc/manual/interactions.html) DESeq2 vignette §Interactions.



- **lfc > 0 (up):** 426 genes, 2.3%
- **lfc < 0 (down):** 244 genes, 1.3%
- **low counts:** 2,527 genes filtered (12%)

## Fold Change Threshold

The default uses  $|\log_2 \text{FC}| > 0$ .  
 Note:  $\log_2 \text{FC} = 1.5$  (a common biological threshold) equals only **0.58** on the  $\log_2$  scale.

## Exporting Results with `write.csv()`

Once you have a results object, export it to the `results/` folder:

```
# Export the interaction results
write.csv(res2, here("results", "Shank3_genoCond_interaction.csv"))

# Export other comparisons
write.csv(res3, here("results", "Shank3_geno_Shank3vsWT_normal.csv"))
write.csv(res4, here("results", "Shank3_geno_Shank3vsWT_sleepDep.csv"))
write.csv(res5, here("results", "Shank3_Cond_SleepDepvsNormal_WT.csv"))
write.csv(res6, here("results", "Shank3_Cond_SleepDepvsNormal_Shank3.csv"))
```

Specify the object to export (e.g., `res2`) and the filename. The script creates `results/` with `dir.create(here("results"))` before the first export.



# MA Plot: plotMA()

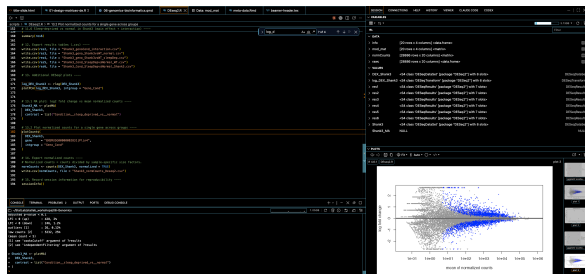


Figure 13: MA plot in Positron — Condition: sleep-deprived vs normal in Shank3 mice

```
log_DEX_Shank3 <- rlog(DEX_Shank3)
plotMA(log_DEX_Shank3)
```

- **x-axis:** mean of normalized counts
- **y-axis:**  $\log_2$  fold change
- **Blue dots:** significantly DE genes ( $p_{adj} < 0.1$ )
- Fan at low counts = noisy LFC estimates in low-abundance genes

# Plotting Single Genes: plotCounts()

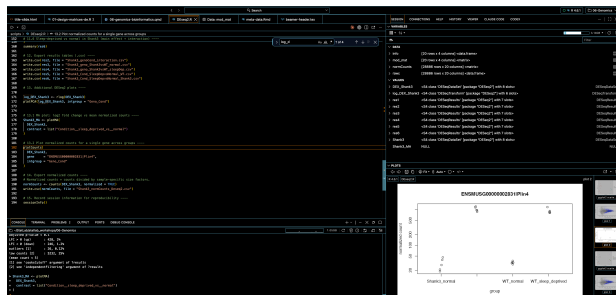


Figure 14: plotCounts for Plin4 across all four groups

```
plotCounts(  
  DEX_Shank3,  
  gene = "ENSMUSG00000002831|Plin4",  
  intgroup = "Geno_Cond"  
)
```

Visualizes normalized counts for a **single gene** split by group. Useful for sanity-checking top DE hits and presenting individual gene results.

## Exporting Normalized Counts

Normalized counts are useful for downstream visualization (heatmaps, clustering) and sharing with collaborators.

```
# Extract normalized counts matrix
normCounts <- counts(DEX_Shank3, normalized = TRUE)

# Save to CSV
write.csv(normCounts, here("results", "Shank3_normCounts_Deseq2.csv"))
```

### Never Use Normalized Counts for DESeq2 Testing

Always provide **raw integer counts** to `DESeqDataSetFromMatrix()`. Normalized counts are for visualization and export only.

## Resources and Further Reading

- **Law et al. (2020)** — *F1000Research*: A guide to creating design matrices for gene expression experiments
- **Love, Huber, Anders (2014)** — *Genome Biology*: the DESeq2 methods paper
- **DESeq2 vignette** — Bioconductor: comprehensive reference with worked examples
- **Holmes & Huber (2019)** — *Modern Statistics for Modern Biology*: free online textbook
- **Bioconductor RNA-seq workflow** — end-to-end analysis from raw data to results

# Thank You!

Thank you for attending! We'd love your feedback.

Scan the QR code to complete a short workshop survey.



Figure 15: Scan for feedback